

**Att&ck and D3fend Full Data Collection with Properties, Updated Nodes, and
Relationships for Unified Cybersecurity Knowledge Graph**

Logan D. Wongbangchuad

Angelo State University

Erdogan Dogdu

05/05/2025

Project Description

Throughout this project, I set out to build a clear, organized ontology for the MITRE ATT&CK and MITRE D3FEND datasets. At the very start, the knowledge graph contained only two generic node types: UcoexMitreAttack and UcoexMitreDefend and neither had any properties beyond a basic URI reference. As the semester progressed, I expanded the attack side of the graph into six distinct node types: techniques, mitigations, tactics, software, campaigns, and groups. Each of these now includes four key properties: ID, name, description, and URL, as well as a domain label. On the defend side, the original node only had the URI. After expansion of the defend nodes, they now have ID, Label, and Description. By the end of the semester, all nodes now have properties and meaningful relationships were created.

DESIGN AND IMPLEMENTATION

For the first two weeks of the semester, I researched my data set to understand the attack and defend data more. Through this research, I learned that the existing knowledge graph was missing five node subtypes from the attack data set. The missing nodes from the knowledge graph were the tactics, campaigns, software's, groups, and mitigations. The two existing nodes in the knowledge graph were the techniques and the defend nodes. Although they were already in the knowledge graph, they did not have any properties to explain what they were, they just had an example URI. Towards the end of my research period, I learned that the two most important tasks I would have to tackle this semester would be adding the missing subtype nodes to the knowledge graph and adding properties to all the nodes. Another major and difficult issue I would learn how to tackle would be creating the relationships between all of the nodes.

To start this journey of adding these nodes to the knowledge graph and adding the properties to the nodes. I first wanted to know how I could add properties to the existing nodes to learn how it's done. I first had to learn what code would need to be adjusted to add these properties to the nodes. I quickly learned to add properties to the nodes I would need to update the .TTL files. Thus began my first challenge. The first data set that was used to create the nodes did not have properties but rather just had the IDs so I had to search for another dataset that contained the full information. The first place I searched was Mitre defend and Mitre attack websites for the correct data. Once I found the correct data set with all the required properties that I would need to insert into the knowledge graph, I knew I would need to use these data sets. Since the new data sets had a different form and more properties than the previous data I used I had to come up with a different way to extract the data, transform it, then load it into the knowledge graph.

MITRE ATT&CK EXTRACTION, TRANSFORMATION, AND LOADING

ATT&CK EXTRACTION

When I initially began the data extraction process, I encountered several notable differences compared to the previous dataset used in the project. The current dataset, which contained all the necessary information required for the knowledge graph, was stored exclusively in Excel file formats, making it a challenge in terms of data extraction. My first task was to determine an efficient method to extract relevant data directly from these Excel files.

Fortunately, I discovered that the Python pandas library provides built-in functionality specifically designed to make the conversion of Excel files into JSON format easier. Using the `pandas.read_excel()` method, I was able to effectively transform the Excel data into JSON structure relatively quickly. However, despite the successful conversion, the output data still contained numerous missing or incomplete values, complicating integration into the existing ontology.

To address this issue, it was essential for me to implement additional data cleaning and parsing techniques. I developed specialized parsing code aimed at identifying, handling, and resolving these incomplete data points. This code involved writing custom functions designed to check for null or missing entries and applying logical rules to populate or remove such entries from the dataset as appropriate. Finally, I called these parsing functions programmatically to refine the JSON data further, ensuring its integrity, completeness, and consistency for the later stages of the project.

Here is the code that extracts the MITRE ATT&CK data:

```
import os
import requests
import json
import pandas as pd
import numpy as np
from io import BytesIO
from config import LOGGER
from process import shared_functions as sf
from bs4 import BeautifulSoup
from fnmatch import fnmatch

# Import your parse functions
from parse import (
    parse_attack_file,
    parse_campaigns_file,
    parse_software_file,
    parse_tactics_file,
    parse_groups_file,
    parse_relationships_file
)

# Per-dataset config: just sheet_name, parse fn, and output name
DATASETS = [
    {
        "name": "attack",
        "sheet_name": None,
        "parse_fn": parse_attack_file,
        "output_json": "attack.json",
    },
    {
        "name": "campaigns",
        "sheet_name": "campaigns",
        "parse_fn": parse_campaigns_file,
        "output_json": "campaigns.json",
    },
    {
        "name": "software",
        "sheet_name": "software",
        "parse_fn": parse_software_file,
        "output_json": "software.json",
    },
    {
        "name": "tactics",
        "sheet_name": "tactics",
        "parse_fn": parse_tactics_file,
        "output_json": "tactics.json",
    },
    {
        "name": "groups",
        "sheet_name": "groups",
        "parse_fn": parse_groups_file,
        "output_json": "groups.json",
    },
    {
        "name": "relationships",
        "sheet_name": "relationships",
        "parse_fn": parse_relationships_file,
        "output_json": "relationships.json",
    },
]
```

```

def download_attack_json_file(cfg):

    url = 'https://attack.mitre.org/resources/attack-data-and-tools/'
    try:
        # Download Excel files from the HTML page and convert them to JSON.
        # Fetch the HTML page from the URL
        html_response = requests.get(url)
        html_response.raise_for_status()
        html = html_response.text

        # Parse HTML using BeautifulSoup
        soup = BeautifulSoup(html, 'html.parser')
        # Base URL is needed to build full links from relative paths.
        base_url = "https://attack.mitre.org"

        # Define the exact Excel file paths we need
        # Dynamically grab whichever versioned Excel files exist
        prefixes = ["enterprise-attack", "mobile-attack", "ics-attack"]
        excel_files = {}
        for a in soup.find_all("a", href=True):
            href = a["href"]
            for prefix in prefixes:
                # match any version, but we'll filter by hyphens next
                pattern = f"/docs/{prefix}-v*/{prefix}-v*.xlsx"
                if fnmatch(href, pattern):
                    # grab just the filename portion
                    filename = href.rsplit("/", 1)[-1]
                    # split on '-' - the "pure" file has exactly 3 parts:
                    # ['mobile', 'attack', 'v*.xlsx']
                    # everything else (matrices, mitigations, ...) has 4+ parts
                    if len(filename.split("-")) == 3:
                        excel_files[prefix] = base_url + href
                    break

        if len(excel_files) < 3:
            LOGGER.info("Not all required Excel links were found!")

        # For each Excel file, download and convert it to JSON
        all_data = []
        for domain, file_url in excel_files.items():
            LOGGER.info(f"Downloading {file_url}")
            file_response = requests.get(file_url)
            file_response.raise_for_status()
            # Use BytesIO to load the Excel content directly
            sheet = cfg["sheet_name"] if cfg["sheet_name"] is not None else 0
            df = pd.read_excel(BytesIO(file_response.content), sheet_name=sheet)
            #df = pd.read_excel(BytesIO(file_response.content), sheet_name=cfg["sheet_name"])
            # Replace any NaN values with None (so they become JSON null)
            df = df.replace({np.nan: None})
            # Convert DataFrame rows to dictionaries and add them to our list.
            records = df.to_dict(orient='records')
            all_data.extend(records)

```

```

# Set the combined JSON data from the Excel files
json_data = {"@graph": all_data}

# Set the filename and volume path
vol_path = os.environ['VOL_PATH']
final_filename = os.path.join(vol_path, cfg["output_json"])

# Check if the file already exists and process accordingly
if os.path.exists(final_filename):
    if sf.check_status(cfg["name"]) == 0:
        LOGGER.info(cfg["output_json"]+" exists...")
        tmp_filename = os.path.join(vol_path, "tmp_" + cfg["output_json"])
        LOGGER.info("Writing tmp_" + cfg["output_json"])
        sf.write_file(tmp_filename, json_data)

        # Calculate the hashes for tmp and final.
        tmp_file_hash = sf.calculate_file_hash(tmp_filename)
        final_file_hash = sf.calculate_file_hash(final_filename)

        # Compare hashes and update if necessary.
        if tmp_file_hash == final_file_hash:
            os.remove(tmp_filename)
            LOGGER.info("The new file is identical to the existing file. Deleted tmp_" + cfg["output_json"] + ".")
        else:
            os.remove(final_filename)
            os.rename(tmp_filename, final_filename)
            LOGGER.info("The new file is different from the existing file. Replaced" + cfg["output_json"] + "with tmp_" + cfg["output_json"] + ".")
    else:
        LOGGER.info(cfg["output_json"] + " DOES NOT exist...")
        LOGGER.info("Writing " + cfg["output_json"])
        sf.write_file(final_filename, json_data)

    LOGGER.info(f"File '{final_filename}' downloaded and saved successfully.")
else:
    # If the file does not exist, simply write the json_data.
    LOGGER.info(cfg["output_json"] + " does not exist. Writing new file.")
    sf.write_file(final_filename, json_data)
    LOGGER.info(f"File '{final_filename}' written successfully.")

LOGGER.info("Beginning JSON data parse for " + cfg["name"])
json_data = cfg["parse_fn"](final_filename)
parsed_filename = "./data/attack/" + cfg["output_json"]
LOGGER.info(f"Beginning JSON data parse save {parsed_filename}")
sf.write_file(parsed_filename, json_data)
LOGGER.info(f"{parsed_filename} saved successfully")
except requests.exceptions.RequestException as e:
    # Handle any API request errors
    LOGGER.info(f"Error making API request: {e}")

```

```

def attack_init():
    LOGGER.info("#####")
    LOGGER.info("Beginning ATTACK Data Downloads")
    LOGGER.info("#####\n")
    for cfg in DATASETS:
        download_attack_json_file(cfg)

    # Now call mapper + ontology just once
    LOGGER.info("ATT&CK Data Download Complete")
    LOGGER.info("Beginning ATT&CK Data Call Mapper Update")
    success = sf.call_mapper_update("attack")
    LOGGER.info("ATTACK Data Call Mapper Update Complete")

    if success:
        LOGGER.info("ATT&CK Data Successfully Mapped")
        LOGGER.info("Beginning ATT&CK Ontology Updater")
        sf.call_ontology_updater(reason=True)
        LOGGER.info("ATT&CK Ontology Updater Complete")
        LOGGER.info("#####\n")
    else:
        LOGGER.info("#####\n")
        LOGGER.error("ATT&CK Ontology Update Failed")
        LOGGER.info("#####\n")

```

Code Breakdown

1. Imports & Constants

- Standard libraries (os, json, requests, BytesIO) plus pandas/numpy for Excel to JSON.
- LOGGER from local config.py for consistent log output.
- Shared helper functions in shared_functions.py (short-lived file I/O, hashing, mapper calls).
- A set of per-sheet configurations (DATASETS) each entry names the sheet, parsing function, and JSON-filename.

2. download_attack_json_file(cfg)

This is called once per dataset in DATASETS.

Key steps:

- a) Scrape the HTML off the MITRE ATT&CK web page for the three core Excel files (enterprise, mobile, ics) by matching link patterns.
- b) Download each matching .xlsx into a pandas DataFrame, replace NAN values, and accumulate rows into one big list.
- c) Wrap that list under "@graph": so it matches the parse functions schema.
- d) Write to the "volume" path under VOL_PATH but only replace the existing file if the new hash differs.
- e) Hand off the final JSON to the dataset's parse_fn function, then write the parsed output into ./data/attack/

3. attack_init()

- Logs start/end of the overall job.
- Iterates through DATASETS, calling download_attack_json_file.
- Once all raw JSON is in place, call the RML mapper (sf.call_mapper_update("attack")) exactly once.
- If mapping succeeds, it calls the ontology updater (sf.call_ontology_updater).

ATT&CK TRANSFORMATION

During the transformation and parsing phase, my primary goal was to generate separate JSON files corresponding to each specific subtype present within the ATT&CK dataset. Structuring the data in this manner was crucial for enabling easier management and ensuring integration into the knowledge graph. For each of these JSON files, it was essential to maintain consistency and completeness. Therefore, I deliberately limited each node to include only five critical properties: ID, name, a comprehensive textual description, the relevant domain, and the associated URL.

Following the creation of these structured JSON files, the next significant step involved loading the prepared data into the knowledge graph. To facilitate this integration, I employed TTL (Turtle) files, which provided a reliable and straightforward mechanism to map each JSON node along with its specific properties directly into the knowledge graph. This structured pipeline, which involved parsing, JSON file creation, and TTL-based integration, ensured that the final dataset was accurately represented, well-structured, and fully aligned with the project's ontology and data integration requirements.

Here is the code that transforms the MITRE ATT&CK data:

```
def parse_attack_file(file_path):  
    parsed_data = []  
  
    try:  
        with open(file_path, 'r') as file:  
            json_data = json.load(file)  
            graph = json_data.get('@graph', [])  
  
            for item in graph:  
                #entry = {'id': item['@id'], 'd3f:attack-id': item['d3f:attack-id'], 'rdfs:label': item['rdfs:label']}  
                entry = {'ID': item['ID'], 'name': item['name'], 'description': item['description'], 'url': item['url'], 'domain': item['domain']}  
                parsed_data.append(entry)  
  
            return parsed_data  
    except Exception as e:  
        # Handle any API request errors  
        LOGGER.info(f"Error parsing ATTACK file: {e}")  
        return None  
  
def parse_mitigations_file(file_path):  
    parsed_data = []  
  
    try:  
        with open(file_path, 'r') as file:  
            json_data = json.load(file)  
            graph = json_data.get('@graph', [])  
  
            for item in graph:  
                entry = {'ID': item['ID'], 'name': item['name'], 'description': item['description'], 'url': item['url'], 'domain': item['domain']}  
                parsed_data.append(entry)  
  
            return parsed_data  
    except Exception as e:  
        # Handle any API request errors  
        LOGGER.info(f"Error parsing ATTACK file: {e}")  
        return None  
  
def parse_campaigns_file(file_path):  
    parsed_data = []  
  
    try:  
        with open(file_path, 'r') as file:  
            json_data = json.load(file)  
            graph = json_data.get('@graph', [])  
  
            for item in graph:  
                entry = {'ID': item['ID'], 'name': item['name'], 'description': item['description'], 'url': item['url'], 'domain': item['domain']}  
                parsed_data.append(entry)  
  
            return parsed_data  
    except Exception as e:  
        # Handle any API request errors  
        LOGGER.info(f"Error parsing ATTACK file: {e}")  
        return None
```

```
def parse_groups_file(file_path):
    parsed_data = []

    try:
        with open(file_path, 'r') as file:
            json_data = json.load(file)
            graph = json_data.get('@graph', [])

            for item in graph:
                entry = {'ID': item['ID'], 'name': item['name'], 'description': item['description'], 'url': item['url'], 'domain': item['domain']}
                parsed_data.append(entry)

            return parsed_data
    except Exception as e:
        # Handle any API request errors
        LOGGER.info(f"Error parsing ATTACK file: {e}")
        return None

def parse_software_file(file_path):
    parsed_data = []

    try:
        with open(file_path, 'r') as file:
            json_data = json.load(file)
            graph = json_data.get('@graph', [])

            for item in graph:
                entry = {'ID': item['ID'], 'name': item['name'], 'description': item['description'], 'url': item['url'], 'domain': item['domain']}
                parsed_data.append(entry)

            return parsed_data
    except Exception as e:
        # Handle any API request errors
        LOGGER.info(f"Error parsing ATTACK file: {e}")
        return None

def parse_tactics_file(file_path):
    parsed_data = []

    try:
        with open(file_path, 'r') as file:
            json_data = json.load(file)
            graph = json_data.get('@graph', [])

            for item in graph:
                entry = {'ID': item['ID'], 'name': item['name'], 'description': item['description'], 'url': item['url'], 'domain': item['domain']}
                parsed_data.append(entry)

            return parsed_data
    except Exception as e:
        # Handle any API request errors
        LOGGER.info(f"Error parsing ATTACK file: {e}")
        return None
```

```
def parse_relationships_file(file_path):
    parsed_data = []

    try:
        with open(file_path, 'r') as file:
            json_data = json.load(file)
            graph = json_data.get('@graph', [])

            for item in graph:
                source_id = item.get('source ID')
                # skip missing or NaN source IDs
                if source_id is None or (isinstance(source_id, float) and math.isnan(source_id)):
                    continue

                # now you know source_id is valid
                entry = {
                    'source ID': source_id,
                    'source type': item.get('source type'),
                    'target ID': item.get('target ID'),
                    'target type': item.get('target type')
                }
                parsed_data.append(entry)

            return parsed_data
    except Exception as e:
        # Handle any API request errors
        LOGGER.info(f"Error parsing ATTACK file: {e}")
        return None
```

Code Breakdown

Parse Attack Files

1. load the file and grab `json_data['@graph']` (an array of full ATT&CK objects).
 2. iterate each item and pick off exactly the five fields we need: ID, name, description, url, domain.
 3. Any I/O or JSON errors are caught, logged, and you return `None` to signal failure.
- * I have one copy of this for each domain (attack, mitigations, campaigns, groups, software, tactics)—identical except the function name.

Parse Relationship file

1. Still read `@graph` but instead of pulling five fixed properties, you only care about the four columns that define your edges.
 2. Guard against missing or NaN source IDs, to avoid garbage relationships.
- * The output is a flat list of four-field dicts that the RML.TTL mapper will use to generate edges.

ATT&CK LOADING

Loading the prepared data into the knowledge graph ultimately emerged as the most challenging aspect of this project. To achieve this integration, I was required to modify the TTL files to accurately incorporate each node into the knowledge graph along with the predefined five essential properties. The complexity of this task primarily stemmed from the nature of TTL files, which proved challenging to implement due to their syntax and the difficulty in diagnosing errors. Specifically, TTL language errors typically do not generate explicit error messages; rather, the absence of expected changes within the knowledge graph served as the only indicator of issues. Consequently, debugging required meticulous review and iterative correction of the TTL code.

Furthermore, effectively addressing these implementation challenges necessitated a thorough understanding of RDF triples, predicate-object relationships, and subject maps, which are fundamental to TTL syntax. Learning these intricate components while simultaneously trying to implement them accurately under time constraints significantly added to the difficulty of the project. Despite these challenges, the iterative process of refining the TTL files, combined with validation against the knowledge graph, enabled successful data loading and integration. This detailed approach ultimately ensured that the final knowledge graph accurately represented the structured dataset with all specified properties correctly implemented.

Here is the code that loads the MITRE ATT&CK data:

```
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix : <http://example.org/rules/> .
@prefix schema: <http://schema.org/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix cve: <http://purl.org/cyber/cve#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix stx: <http://purl.org/cyber/stx#> .
@prefix uco: <http://purl.org/cyber/uco#> .
@prefix xml: <http://www.w3.org/XML/1998/namespace> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix capec: <http://purl.org/cyber/capec#> .
@prefix ucoex: <http://example.com/ucoex#> .
@base <http://purl.org/cyber/uco#> .

<#ATTACKMap> a rr:TriplesMap;
  rml:logicalSource [
    rml:source "../data/attack/attack.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator "$[*]"
  ];
  rr:subjectMap [
    rr:template "http://example.com/ucoex#{ID}";
    rr:class <http://example.com/ucoexMITREATTACK>
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexNAME>;
    rr:objectMap [
      rml:reference "name";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDESCRIPTION>;
    rr:objectMap [
      rml:reference "description";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexURL>;
    rr:objectMap [
      rml:reference "url";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDOMAIN>;
    rr:objectMap [
      rml:reference "domain";
      rr:datatype xsd:string
    ]
  ];
].
```

```

<#MITIGATIONSMap> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "/data/attack/mitigations.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator        "$[*]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{ID}";
    rr:class              <http://example.com/ucoexMITIGATIONS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexNAME>;
    rr:objectMap [ rml:reference "name"; rr:datatype xsd:string ]
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexDESCRIPTION>;
    rr:objectMap [ rml:reference "description"; rr:datatype xsd:string ]
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexURL>;
    rr:objectMap [ rml:reference "url"; rr:datatype xsd:string ]
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexDOMAIN>;
    rr:objectMap [ rml:reference "domain"; rr:datatype xsd:string ]
  ].

```

```

<#CAMPAIGNSMap> a rr:TriplesMap;
  rml:logicalSource [
    rml:source "/data/attack/campaigns.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator "$[*]"
  ];
  rr:subjectMap [
    rr:template "http://example.com/ucoex#{ID}";
    rr:class <http://example.com/ucoexCAMPAIGNS>
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexNAME>;
    rr:objectMap [
      rml:reference "name";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDESCRIPTION>;
    rr:objectMap [
      rml:reference "description";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexURL>;
    rr:objectMap [
      rml:reference "url";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDOMAIN>;
    rr:objectMap [

```

```

<#GROUPSMap> a rr:TriplesMap;
  rml:logicalSource [
    rml:source "../data/attack/groups.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator "$[*]"
  ];
  rr:subjectMap [
    rr:template "http://example.com/ucoex#{ID}";
    rr:class <http://example.com/ucoexGROUPS>
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexNAME>;
    rr:objectMap [
      rml:reference "name";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDESCRIPTION>;
    rr:objectMap [
      rml:reference "description";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexURL>;
    rr:objectMap [
      rml:reference "url";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDOMAIN>;
    rr:objectMap [
      rml:reference "domain";
      rr:datatype xsd:string
    ]
  ];
].

```

```

<#SOFTWAREMap> a rr:TriplesMap;
  rml:logicalSource [
    rml:source "../data/attack/software.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator "$[*]"
  ];
  rr:subjectMap [
    rr:template "http://example.com/ucoex#{ID}";
    rr:class <http://example.com/ucoexSOFTWARE>
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexNAME>;
    rr:objectMap [
      rml:reference "name";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexDESCRIPTION>;
    rr:objectMap [
      rml:reference "description";
      rr:datatype xsd:string
    ]
  ];
].

```



```
<#TACTICSMap> a rr:TriplesMap;  
  rml:logicalSource [  
    rml:source "../data/attack/tactics.json";  
    rml:referenceFormulation ql:JSONPath;  
    rml:iterator "$[*]"  
  ];  
  rr:subjectMap [  
    rr:template "http://example.com/ucoex#{ID}";  
    rr:class <http://example.com/ucoexTACTICS>  
  ];  
  rr:predicateObjectMap [  
    rr:predicate <http://example.com/ucoexNAME>;  
    rr:objectMap [  
      rml:reference "name";  
      rr:datatype xsd:string  
    ]  
  ];  
  rr:predicateObjectMap [  
    rr:predicate <http://example.com/ucoexDESCRIPTION>;  
    rr:objectMap [  
      rml:reference "description";  
      rr:datatype xsd:string  
    ]  
  ];  
  rr:predicateObjectMap [  
    rr:predicate <http://example.com/ucoexURL>;  
    rr:objectMap [  
      rml:reference "url";  
      rr:datatype xsd:string  
    ]  
  ];  
  rr:predicateObjectMap [  
    rr:predicate <http://example.com/ucoexDOMAIN>;  
    rr:objectMap [  
      rml:reference "domain";  
      rr:datatype xsd:string  
    ]  
  ]  
];
```

Code Breakdown

1. Prefix declarations (at the top of the file):

- * Standard RML/R2RML prefixes (rml:, rr:, ql:) plus the custom (ucoex:)
- * These let you use compact terms like rr:TriplesMap and ucoexDOMAIN.

2. Node TriplesMaps

For each JSON file under ./data/attack/.json, use subject map to create the nodes and insert them into the knowledge graph and use four predicateObjectMap blocks for name, description, url, domain to insert properties into the nodes

Repeat this pattern for:

<#MITIGATIONSMap> reading mitigations.json, class ucoexMITIGATIONS

<#CAMPAIGNSMap> reading campaigns.json, class ucoexCAMPAIGNS

<#GROUPSMap> reading groups.json, class ucoexGROUPS

<#SOFTWAREMap> reading software.json, class ucoexSOFTWARE

<#TACTICSMap> reading tactics.json, class ucoexTACTICS

RELATIONSHIPS

After successfully creating the new nodes with their respective properties, the subsequent critical step involved establishing relationships among these nodes within the knowledge graph.

Fortunately, the MITRE ATT&CK dataset already included an existing data file explicitly detailing the relationships, greatly simplifying my task. My primary responsibility, therefore, was to effectively transfer and integrate these predefined relationships into the knowledge graph.

To accomplish this, I first had to extract the necessary relationship data from the original Excel file provided by MITRE. Following extraction, I undertook a transformation and parsing process specifically tailored to isolate and refine the data to retain only essential information: the source

ID and target ID for each relationship. With this streamlined relationship data prepared, I proceeded to modify the corresponding TTL file to establish the actual connections between the existing nodes.

However, during this process, I encountered a significant challenge. Initially, the TTL implementation inadvertently created entirely new nodes rather than utilizing the existing nodes within the knowledge graph. Addressing this issue required me to thoroughly analyze the TTL syntax and mapping methods. Eventually, I identified a method to adjust the TTL configuration so that it would accurately reference and map to existing nodes instead of generating duplicate or extraneous ones. This corrective measure involved careful revisions of subject and object mappings within the TTL file, ultimately resolving the problem. Through adjustments and validation, I successfully ensured that the relationships accurately reflected the intended connections between the pre-existing nodes, significantly enhancing the structural integrity and accuracy of the knowledge graph.

Here is the code that creates the relationship edges in the knowledge graph:

```
<#MitigationToTechnique> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "/data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator         "$[?(@['source type']=='mitigation' && @['target type']=='technique')]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{source ID}";
    rr:class              <http://example.com/ucoexMITIGATIONS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexMITIGATES>;
    rr:objectMap [
      rr:parentTriplesMap <#ATTACKMap>;
      rr:joinCondition [
        rr:child          "target ID";
        rr:parent          "ID"
      ]
    ]
  ];
].

<#CampaignToGroup> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "/data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator         "$[?(@['source type']=='campaign' && @['target type']=='group')]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{source ID}";
    rr:class              <http://example.com/ucoexCAMPAIGNS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexATTRIBUTEDTO>;
    rr:objectMap [
      rr:parentTriplesMap <#GROUPSMap>;
      rr:joinCondition [
        rr:child          "target ID";
        rr:parent          "ID"
      ]
    ]
  ];
].

<#CampaignToSoftware> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "/data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator         "$[?(@['source type']=='campaign' && @['target type']=='software')]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{source ID}";
    rr:class              <http://example.com/ucoexCAMPAIGNS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexCampaignUsesSoftware>;
    rr:objectMap [
      rr:parentTriplesMap <#SOFTWAREMap>;
      rr:joinCondition [
        rr:child          "target ID";
        rr:parent          "ID"
      ]
    ]
  ];
].
```

```

<#CampaignToTechnique> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "./data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator         "$[?(@['source type']=='campaign' && @['target type']=='technique')]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{source ID}";
    rr:class              <http://example.com/ucoexCAMPAIGNS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexCampaignUsesTechnique>;
    rr:objectMap [
      rr:parentTriplesMap <#ATTACKMap>;
      rr:joinCondition [
        rr:child          "target ID";
        rr:parent          "ID"
      ]
    ]
  ]
].

<#GroupToSoftware> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "./data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator         "$[?(@['source type']=='group' && @['target type']=='software')]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{source ID}";
    rr:class              <http://example.com/ucoexGROUPS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexGroupUsesSoftware>;
    rr:objectMap [
      rr:parentTriplesMap <#SOFTWAREMap>;
      rr:joinCondition [
        rr:child          "target ID";
        rr:parent          "ID"
      ]
    ]
  ]
].

<#GroupToTechnique> a rr:TriplesMap;
  rml:logicalSource [
    rml:source          "./data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator         "$[?(@['source type']=='group' && @['target type']=='technique')]"
  ];
  rr:subjectMap [
    rr:template          "http://example.com/ucoex#{source ID}";
    rr:class              <http://example.com/ucoexGROUPS>
  ];
  rr:predicateObjectMap [
    rr:predicate          <http://example.com/ucoexGroupUsesTechnique>;
    rr:objectMap [
      rr:parentTriplesMap <#ATTACKMap>;
      rr:joinCondition [
        rr:child          "target ID";
        rr:parent          "ID"
      ]
    ]
  ]
].

```

```

<#SoftwareToTechnique> a rr:TriplesMap;
  rml:logicalSource [
    rml:source                "./data/attack/relationships.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator               "$[?(@['source type']=='software' && @['target type']=='technique')]"
  ];
  rr:subjectMap [
    rr:template                "http://example.com/ucoex#{source ID}";
    rr:class                   <http://example.com/ucoexSOFTWARE>
  ];
  rr:predicateObjectMap [
    rr:predicate               <http://example.com/ucoexSoftwareUsesTechnique>;
    rr:objectMap [
      rr:parentTriplesMap      <#ATTACKMap>;
      rr:joinCondition [
        rr:child               "target ID";
        rr:parent               "ID"
      ]
    ]
  ]
].

```

Code Breakdown

1. Edge TriplesMaps

- *For each relationship row in relationships.json, add standalone maps:
- *Use rml:iterator to specify which source type and target type to use in the json
- *Use subject map to specify the source node to use
- *Use predicate object map to specify the relationship between the two nodes
- *Use parentTriplesMap to specify which node the source node should map to.

Follow the same structure for:

- <#CampaignToGroup>
- <#CampaignToSoftware>
- <#CampaignToTechnique>
- <#GroupToSoftware>
- <#GroupToTechnique>
- <#SoftwareToTechnique>

MITRE D3FEND EXTRACTION, TRANSFORMATION, AND LOADING

D3FEND EXTRACTION

In the extraction process for the `defend_collection.py` script, minimal modifications were required as the original code handled the collection and extraction. The primary adjustment involved updating the URL to ensure it referenced the correct endpoint, which provided the necessary data properties in JSON format. The previously implemented URL was insufficient as it only contained the defend IDs without any additional descriptive properties. This URL modification was essential because accessing JSON data allowed the retrieval of detailed information which enhanced the accuracy and completeness of the dataset extracted. This update streamlined data processing steps and contributed to improved data for later entry into the knowledge graph.

Here is the code that extracts the mitre d3fend data

```
import os
import requests
from config import LOGGER
from parse import parse_d3fend_file
from process import shared_functions as sf
# from utilities import check_status, write_file, calculate_file_hash, call_mapper_update, call_ontology_update

def download_d3fend_json_file():

    url = 'https://d3fend.mitre.org/ontologies/d3fend.json'

    try:
        # Make API call to retrieve JSON data
        response = requests.get(url)
        response.raise_for_status() # Raise an exception for HTTP errors
        json_data = response.json()

        # Set the filename
        final_filename = os.path.join(os.environ['VOL_PATH'], "d3fend.json")

        # Check if the request was successful
        if response.status_code == 200:
            if sf.check_status("d3fend") == 0:
                LOGGER.info("d3fend.json exists...")
                filename = os.path.join(os.environ['VOL_PATH'], "tmp_d3fend.json")
                LOGGER.info("Writing tmp_d3fend.json")
                sf.write_file(filename, json_data)

                # Calculate the hashes for tmp and final.
                tmp_file_hash = sf.calculate_file_hash(filename)
                final_file_hash = sf.calculate_file_hash(final_filename)

                # Compare hashes
                if tmp_file_hash == final_file_hash:
                    # Hashes are the same, delete tmp file
                    os.remove(filename)
                    LOGGER.info("The new file is identical to the existing file. Deleted tmp_d3fend.json.")
                else:
                    # Hashes are different, replace existing file
                    os.remove(final_filename)
                    os.rename(filename, "/vol/data/d3fend.json")
                    LOGGER.info("The new file is different from the existing file. Replaced d3fend.json with "
                                "tmp_d3fend.json.")
            else:
                LOGGER.info("d3fend.json DOES NOT exist...")
                filename = os.path.join(os.environ['VOL_PATH'], "d3fend.json")
                LOGGER.info("Writing d3fend.json")
                sf.write_file(filename, json_data)

            LOGGER.info(f"File '{filename}' downloaded and saved successfully.")
        else:
            LOGGER.info("Failed to download the D3FEND JSON file.")

        LOGGER.info("Beginning JSON data parse for d3fend")
        d3fend_json_data = parse_d3fend_file(final_filename)
        d3fend_parsed_filename = "./data/d3fend/d3fend.json"
        LOGGER.info(f"Beginning JSON data parse save {d3fend_parsed_filename}")
        sf.write_file(d3fend_parsed_filename, d3fend_json_data)
        LOGGER.info(f"{d3fend_parsed_filename} saved successfully")
    except requests.exceptions.RequestException as e:
        # Handle any API request errors
        LOGGER.info(f"Error making API request: {e}")
```



```
def d3fend_init():
    LOGGER.info("#####")
    LOGGER.info("Beginning D3FEND Data Download")
    LOGGER.info("#####\n")

    # Download latest D3FEND json from mitre.
    download_d3fend_json_file()

    LOGGER.info("D3FEND Data Download Complete")
    LOGGER.info("Beginning D3FEND Data Call Mapper Update")

    successfully_mapped = sf.call_mapper_update("d3fend")

    if successfully_mapped:
        LOGGER.info("D3FEND Data Successfully Mapped")
        LOGGER.info("Beginning D3FEND Ontology Updater")
        sf.call_ontology_updater(reason=True)
        LOGGER.info("#####\n")
        LOGGER.info("D3FEND Ontology Updater Complete")
        LOGGER.info("#####\n")
    else:
        LOGGER.info("#####")
        LOGGER.info("D3FEND Ontology Update Failed")
        LOGGER.info("#####\n")
```

Code Breakdown

1. Imports & Dependencies

- * parse_d3fend_file: turns the raw JSON into your simplified data model.
- * shared_functions (sf)

2. download_d3fend_json_file()

- * GET <https://d3fend.mitre.org/ontologies/d3fend.json>
- * If `sf.check_status("d3fend") == 0` → the file already exists.
- * Write a temp file (tmp_d3fend.json).

- * Compute hashes on temp vs. final.
- * If identical → delete temp; otherwise rename temp → final.
- * Else (first time) → write straight to d3fend.json.

Parse step

- * Calls `parse_d3fend_file(final_filename)` once the raw JSON is settled.
- * Writes the parsed output to `d3fend.json` (your downstream RML input).

3. `d3fend_init()`

- * Startup logs marking the beginning of the D3FEND job.
- * Calls `download_d3fend_json_file()` to refresh raw + parsed JSON.
- * `sf.call_mapper_update("d3fend")` invokes your RML loader against the new data.
- * `sf.call_ontology_updater(reason=True)` pushes any ontology deltas into Neo4j.

D3FEND TRANSFORMATION

To accurately transform the JSON data into the required format and include the necessary properties, modifications were made to the `parse_defend_file` function within the `parse.py` script. Initially, the defend parsing function was limited to processing JSON files that only contained defend IDs and labels. I expanded this parsing functionality to include additional properties, specifically ID, name, definition, and label. I also integrated an API call within the parsing function to retrieve the corresponding attack relationships for each defend entry. This relationship data was appended to each JSON entry. If a defend node lacked an associated attack node, the parsing function was programmed to display an explicit error message. After successfully generating entries with complete property sets and accurate mappings to attack nodes, the modified and enriched JSON file was returned to the `attack_collection` script for further processing.

Here is the code that transforms the JSON data:

```
def parse_d3fend_file(file_path):

    parsed_data = []

    try:
        with open(file_path, 'r') as file:
            json_data = json.load(file)
            for item in json_data['@graph']:
                if item.get("d3f:d3fend-id") is None:
                    continue
                entry = {'@id': item.get('@id', ''), 'd3f:definition': item.get('d3f:definition', ''),
                        'd3f:d3fend-id': item.get('d3f:d3fend-id', ''),
                        'rdfs:label': item.get('rdfs:label', '')}
                api_url = f"https://d3fend.mitre.org/api/technique/{item['@id']}.json"

                try:
                    # Make API call to retrieve JSON data
                    response = requests.get(api_url)
                    response.raise_for_status() # Raise an exception for HTTP errors
                    json_data = response.json()

                    # Parse JSON data for off_tech_id
                    bindings = json_data['def_to_off']['results']['bindings']
                    off_tech_id = ""
                    for binding in bindings:
                        off_tech_id = binding.get('off_tech_id', {}).get('value', '')
                        if off_tech_id:
                            break # Assuming there's only one off_tech_id
                    entry['off_tech_id'] = off_tech_id
                except requests.exceptions.RequestException as e:
                    # Handle any API request errors
                    continue # Skip this entry if API call fails

                parsed_data.append(entry)
            return parsed_data
    except Exception as e:
        # Handle any API request errors
        LOGGER.info(f"Error parsing D3FEND file: {e}")
        return None
```

Code Breakdown

1. Grab only items with a d3f:d3fend-id in the very messy JSON file.
2. assemble a dict of four core properties: ID, name, definition, and label.
3. Call out to <https://d3fend.mitre.org/api/technique/{@id}.json> to pull the off_tech_id that ties D3FEND nodes to ATT&CK techniques.
4. Any missing mappings in the API call skips the addition of the mapping for that entry, but doesn't stop the whole function.

D3FEND LOADING

To effectively load the nodes into the knowledge graph, modifications were made to the defend_rml.ttl file. These adjustments included updating the subject map configuration to accurately map nodes into the database and defining properties for each node. Additionally, the relationships between the defend nodes and corresponding attack techniques were explicitly established within this file. Overall, the implementation of these enhancements was relatively straightforward, presenting minimal challenges and contributed to the accurate representation of data within the knowledge graph.

Here is the code that loads the nodes into the knowledge graph:

```
@prefix rml: <http://semweb.mmlab.be/ns/rml#> .
@prefix rr: <http://www.w3.org/ns/r2rml#> .
@prefix ql: <http://semweb.mmlab.be/ns/ql#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix : <http://example.org/rules/> .
@prefix schema: <http://schema.org/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix cve: <http://purl.org/cyber/cve#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix stx: <http://purl.org/cyber/stx#> .
@prefix uco: <http://purl.org/cyber/uco#> .
@prefix xml: <http://www.w3.org/XML/1998/namespace> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix capec: <http://purl.org/cyber/capec#> .
@prefix ucoex: <http://example.com/ucoex#> .
@prefix d3f: <http://d3fend.mitre.org/ontologies/d3fend.owl#> .
@base <http://purl.org/cyber/uco#> .

<#DefendMapping> a rr:TriplesMap;
  rml:logicalSource [
    rml:source "/data/d3fend/d3fend.json";
    rml:referenceFormulation ql:JSONPath;
    rml:iterator "$[*]"
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexMITRED3FEND_ID>;
    rr:objectMap [
      rml:reference "d3f:d3fend-id";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexMITRED3FEND_DEFINITION>;
    rr:objectMap [
      rml:reference "d3f:definition";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexMITRED3FEND_LABEL>;
    rr:objectMap [
      rml:reference "rdfs:label";
      rr:datatype xsd:string
    ]
  ];
  rr:predicateObjectMap [
    rr:predicate <http://example.com/ucoexhasMITREATTACK>;
    rr:objectMap [
      rr:template "http://example.com/ucoex#{off_tech_id}";
      rr:class <http://example.com/ucoexMITREATTACK>
    ]
  ];
  rr:subjectMap [
    rr:template "http://example.com/ucoex#{@id}";
    rr:class <http://example.com/ucoexMITRED3FEND>
  ].
```

Code Breakdown

1. TriplesMap declaration

- `<#DefendMapping> a rr:TriplesMap;` – Defines one mapping that will process every element in `d3fend.json`.

2. Logical Source

- `rml:source "./data/d3fend/d3fend.json"`: points at your parsed D3FEND file.
- `ql:JSONPath + rml:iterator "$[*]":` iterate over each top-level array entry.

3. Property maps (predicateObjectMap)

There are four blocks that turn JSON fields into RDF literals:

– D3FEND-ID

- Predicate: `ucoexMITRED3FEND_ID`

– Definition

- Predicate: `ucoexMITRED3FEND_DEFINITION`

– Label

- Predicate: `ucoexMITRED3FEND_LABEL`

– Link to ATT&CK

- Predicate: `ucoexhasMITREATTACK`
- ObjectMap uses a template:

```
rr:template "http://example.com/ucoex#{off_tech_id}" rr:class
<http://example.com/ucoexMITREATTACK>
```

This creates the outgoing edge to the ATT&CK technique whose ID was fetched as `off_tech_id` during parsing.

4. Subject map

- `rr:subjectMap`

- `rr:template "http://example.com/ucoex#{@id}":` each JSON object's `@id` becomes the local part of your D3FEND-entity URI

OPTIMIZATION

Upon successfully integrating all necessary data and establishing accurate relationships within the knowledge graph, the final step required was optimizing the underlying codebase. Initially, the implementation involved around 14 separate files—to carry out the data collection, parsing, and insertion operations. Although functional, this approach negatively impacted code maintainability, readability, and efficiency.

Recognizing the importance of streamlined and maintainable code, I started my optimization process. My goal was to significantly condense and restructure the codebase, integrating functionality into fewer, more cohesive scripts. To achieve this, I consolidated related operations into centralized scripts, resulting in clearly defined, dedicated files: one responsible for data extraction and parsing, and another specifically designed for loading nodes and relationships into the knowledge graph.

Ultimately, this focused optimization effort effectively reduced the total number of script files from 14 to 4. This substantial reduction greatly enhanced the clarity, structure, and readability of the codebase. Moreover, it facilitated easier debugging, simplified future maintenance tasks, and improved overall performance, thereby ensuring the project's sustainability and robustness in ongoing and future implementations.

FUTURE WORK

Although significant progress has been achieved, several areas remain for future enhancements to the knowledge graph. Notably, a few critical relationships still need to be properly mapped and integrated. Specifically, I did not have enough time to establish relationships between mitigation nodes and defend nodes within the knowledge graph, and these remain unresolved. Addressing this gap is essential to fully capturing the interconnected nature of the ATT&CK and D3FEND datasets. Another area that would need to be improved in the future is optimizing the parse.py file. In the parse file, I created 6 separate functions to parse each dataset. I believe consolidating each function into one parse function to parse the attack dataset as a whole would be beneficial to increase the readability and efficiency of the code.

References

<https://neo4j.com/docs/python-manual/current/>

<https://github.com/edogdu/UCKG>

<https://graphacademy.neo4j.com/categories/>

<https://graphacademy.neo4j.com/courses/cypher-fundamentals/>

<https://graphacademy.neo4j.com/courses/llm-knowledge-graph-construction/>

<https://graphacademy.neo4j.com/courses/gds-product-introduction/>